

Lab Exercise 6: Python and Pandas

Due: April 1, 2026 on Canvas

The purpose of this exercise is to practice exploring and cleaning data. Feel free to work in your project groups! If you work together, please only submit one assignment per team. You can answer all the questions in markdown cells in your notebook.

1. Load the San Mateo AirBnB dataset into your notebook as a pandas dataframe. The data are from <https://insideairbnb.com/>

- How many rows and columns do you have?
- What are the column names?
- What data types appear in the dataset?
- Are there missing values? (hint: you can answer all these questions using the `.info()` function.)

2. Generate descriptive statistics for the columns.

3. Missing values:

- Find the number of missing values for each column. You can use the code `df.isna().sum().sort_values(ascending=False)` to do this.
- Find the % missing values `(df.isna().mean() * 100).sort_values(ascending=False)`
- Which columns have the most missing values? Does these seem important to deal with?

4. Duplicate rows

- How many duplicate rows are there? You can use `df.duplicated().sum()` to find out.
- If there are duplicated ones, you can drop them using `df = df.drop_duplicates()`.

5. Get information on some categorical variables:

```
for col in ['neighbourhood_group', 'neighbourhood', 'room_type']:
    if col in df.columns:
        print(f'Column: {col}')
        print(df[col].value_counts(dropna=False).head(10))
```

What does this tell you?

6. Now, let's see about cleaning the price data a little. What do you think is a reasonable upper-bound for the price? Let's assume that anything over \$1000 is not relevant for our analysis. Before you do any analysis, create a copy of your dataframe so you can always go back to the original data if something goes wrong: `df_clean = df.copy()`. (Using only `df_clean = df` will not create a real copy of the dataframe. It creates a so-called "view.") Notice that the code below makes sure that the `price` column exists before trying to do any operations on it. This will allow you to reuse your code for a different city's dataset that may not contain all the same information.

```
# Example: remove rows with missing price, if the column exists
if 'price' in df_clean.columns:
    df_clean = df_clean[df_clean['price'].notna()]

# Check result
df_clean.shape

# Example: look at price distribution numerically
if 'price' in df_clean.columns:
    print(df_clean['price'].describe())

# Example: filter extreme prices
# Adjust the upper limit if needed based on your dataset.
if 'price' in df_clean.columns:
    df_clean = df_clean[(df_clean['price'] > 0) & (df_clean['price'] <=
1000)]

# Check result
if 'price' in df_clean.columns:
    print(df_clean['price'].describe())
```

7. What do the prices look like? You can use the code below to help you answer the question.

```
if 'price' in df_clean.columns:
    plt.hist(df_clean["price"], bins=30)

# the code below will give the same output! pandas also has a
# .plot() function, with slightly different syntax
#df_clean['price'].plot(kind='hist', bins=30)
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.title('Distribution of Airbnb Prices')
plt.show()
```

8. Do room types have different average prices? You can use the code below to help you answer the question.

```
if {'room_type', 'price'}.issubset(df_clean.columns):
    room_price =
df_clean.groupby('room_type')['price'].mean().sort_values(ascending=False)
```

```
room_price

if {'room_type', 'price'}.issubset(df_clean.columns):
    room_price.plot(kind='bar')
    plt.xlabel('Room Type')
    plt.ylabel('Average Price')
    plt.title('Average Price by Room Type')
    plt.xticks(rotation=45)
    plt.show()
```

Note that, this time, we are using the `pandas plot` function, because it is just easier in this context. Interesting side quest: how would you do this using the `matplotlib` version of bar charts?

9. Which areas are most expensive? You can use the code below to help you answer the question.

```
if {'neighbourhood_group', 'price'}.issubset(df_clean.columns):
    area_price =
df_clean.groupby('neighbourhood_group')['price'].mean().sort_values(ascending=False)
    area_price
elif {'neighbourhood', 'price'}.issubset(df_clean.columns):
    area_price =
df_clean.groupby('neighbourhood')['price'].mean().sort_values(ascending=False).head(10)
    area_price
```

10. Is there a relationship between reviews and price?

```
if {'number_of_reviews', 'price'}.issubset(df_clean.columns):
    df_clean.plot(kind='scatter', x='number_of_reviews', y='price')
    plt.title('Price vs Number of Reviews')
    plt.show()
```

Bonus point: Rerun your code using the Oakland Airbnb dataset. What differences do you see between the San Mateo area and Oakland?